Indexes trade off faster retrieval speed for more disk space consumption. When performance is critical it may be worth creating specialized indexes.

As a brief introduction to specialized indexes, “Multicolumn Indexes” and “Covering Indexes” are specialized index types that can be used as part of your query optimization tactics.

The query planner may scan the entire table as a Sequential Scan or may use an Index and perform an Index Scan. Is there a middle ground between table scans and Index scans? There is – Bitmap Scans split the difference.

EXPLAIN Scan Nodes and Bitmap Scans

Let’s see how Bitmap Scans are used. The *Database Page Layout*[17] page is extensive and beyond the scope of the chapter, but some basics will be covered here.

PostgreSQL table data (the *heap*) is represented in a *Page*. The bitmap PostgreSQL builds is one it creates on the fly by visiting every Page, and marking whether a Page contains a row and should be scanned or not.

The scan type is something PostgreSQL chooses but is exposed in the query execution plan.

Bitmap Scans can refer to either the Heap (table) or an Index. A Bitmap Index Scan (See: *EXPLAIN - Bitmap Index Scan*[18]) is like a middle ground between a slow scan of the entire table and a fast Index Scan. When would a Bitmap Index Scan be used and not an Index Scan? One scenario would be when an Index would be useful but it doesn’t exist covering the fields being filtered on.

On the other side of things with very large result sets, PostgreSQL may choose to not use an Index Scan for an index that exists because it might be faster to scan the entire table before filtering.

A Bitmap Heap Scan builds a bitmap data structure from a heap scan. A Bitmap Index Scans builds on top of that.

The bitmap is one bit per heap page. (See: *Understanding "bitmap heap scan" and "bitmap index scan"*[19])

17. https://www.postgresql.org/docs/current/storage-page-layout.html
18. https://pganalyze.com/docs/explain/scan-nodes/bitmap-index-scan
19. https://dba.stackexchange.com/a/119391